

Feedback Representation Is Not the Bottleneck: A Small LLM Game Designer Picks Right, Edits Wrong

Alaz Cig

Contents

- Abstract
- Introduction
- Methods
 - The design loop
 - The skill-edge objective
 - Verifying the skill ladder
 - The simulator and the editor
 - Matched feedback representation experiment
 - The chain of experiments
- Results
 - Finding 1 – More information does not help and the curve does not rescue it
 - Finding 2 – The smaller model's apparent successes are broken games, caught by the validity check
 - Finding 3 – The model knows the right change but cannot execute it
 - Finding 4 – The failure mechanism remains after abstracting the game away
 - Finding 5 – The task itself is feasible: a scripted rule built from the model's own advice reaches the target
 - Finding 6 – The right edit exists in the model's own sampled candidates and is lost at the moment of commitment
 - Finding 7 – Better selection and more edit attempts do not repair construction
 - Finding 8 – The designer succeeds once the model splits responsibility with the harness
- Discussion
- Conclusion
- References

Abstract

Tuning a game is slow because the only definitive test is playthroughs: a designer changes a rule, watches who wins and why, and changes the rule again. Simulated playtesters make the playing cheap; what remains is to distill their gameplay into a metric worth optimizing, and to represent what the playtesters experienced in a form a language model can act on, edit after edit, in a closed loop. This project builds that loop and investigates what limits it. A small language model edits a Monopoly board exposed as 21 continuous knobs. Its task is to steer the skill-edge score – how much skill matters, measured as the share of wealth an expert simulated playtester wins against a random one – from the default game’s 0.73 to a target of 0.60 (zone 0.57–0.63) within eight edits. Games must also remain skill-order valid: playtesters with wider action spaces still outperform those with narrower ones or random play. The hypothesis had two parts: more information would not help the designer, and a progress curve would beat a table of numbers. The first was found true; the second was falsified: none of the reports, rich or sparse, curve or table, lowers the score toward the target. None of 9 trials per reporting type on the larger model reaches the zone, and receiving no feedback does as well as receiving everything. The model knows the right change: asked to pick the best of four hand-tuned boards, it picks a skill-edge-optimizing board in 24 of 24 trials and explains why, yet repairs a bad board in 0 of 5 trials. The failure mode remains after replacing the game with a synthetic problem. The task itself is shown to be feasible: a scripted rule built from the model’s own stated advice, with no active model, reaches the target in 8 of 9 trials. The right edits exist among the model’s own sampled candidates – the final answer it commits to is worse than 92% of its own 50 samples – and neither simulator-assisted selection nor a doubled edit budget repairs the failure. The solution is reassigning construction to the harness around the model: the model only names the knob and the edit direction, and the harness determines the edit magnitude and decides when to stop. With the harness, the designer succeeds in 9 of 9 trials in a median of two edits given a categorical grouping of knobs. It still succeeds when shown the ungrouped raw list of all 21 knobs (7 of 9 trials), and fails when the list is withheld and the change must be described in free prose then acted on by string matching (0 of 9 trials). A small language model is a competent chooser but not a competent constructor of edits; a working design loop must respect that division of labor.

Introduction

Tuning a game is slow because the only definitive test is playthroughs. A designer changes a rule, runs a playtest, watches who wins and why, and changes the rule again. The loop is bottlenecked by the expensive and slow question of “does this version play well.” Simulated playtesters remove that cost: hand-coded computer players can cheaply and rapidly play many games per rule change. Cheap playthroughs alone, however, do not close the design loop. Two questions remain to incorporate the playtesters in a working loop. The first is measurement: a property like how much skill matters must be distilled from a pile of multiplayer outcomes into a single metric a loop can optimize. And in a multiplayer game, who is expected to win against whom and how

often is a design decision itself. The second is representation: the playtesters' experiences must be turned into feedback that a small language model, as the designer, can act on through multiple edits in a closed loop. This project builds that loop and investigates which part limits it.

The game being tuned is a Monopoly board, exposed as a set of adjustable knobs: a rent multiplier and a cost multiplier for each property group, plus the player salary, totaling 21 continuous knobs. Each design round, the model reads a short feedback report on how a fixed set of players fared on the current board and returns a small edit to those knobs; the edited board is re-simulated and the cycle repeats for a budget of eight edits. The model is asked to act as the optimizer; the simulated players are its measuring instrument. The constraints are fixed throughout: a small open model, eight edits per trial, and feedback that arrives only through the simulated players' noisy measurements.

A small model in the designer's role can fail for two reasons. It might not be told enough: an information limit, curable by a richer report. Or it might be told enough and still be unable to act: a search-and-control limit, immune to extra information. An early pilot run hinted at a specific version of the first explanation: more information did not help, and what seemed to matter was the form the feedback took. The hypothesis is that more information will not help the designer and that presenting the same feedback as a progress curve (a picture of how the skill-edge score varies across a game) will beat presenting it as a table of numbers. If the experiments cleanly separate "the limit is the information or its format" from "the limit is the model's ability to act," the project succeeds, ideally with a remedy. It fails if confounds or noise leave the two explanations indistinguishable with no path to a remedy. To credit an effect to the form of the feedback, the reports must carry identical information and differ only in presentation – and must leave the model an identical ability to act.

This report follows the feedback theory from diagnosis to repair. The diagnosis is that the bottleneck is not information or its format (every report, including no report at all, leaves the score outside the zone), and it is not knowledge: the model identifies the correct knob and direction and explains why, yet cannot execute the change on the real game or on a synthetic problem with the Monopoly game removed. The bottleneck is found to be at the final step: the task is solvable with the model's own stated advice and improving edits exist among the model's own sampled candidates but are lost at the moment of commitment. Neither simulator-assisted selection nor a doubled edit budget repairs the failure. Based on the diagnosis, the repair is to remove the responsibility of construction from the model. When the model only names the knob and the edit direction, and the harness determines the edit magnitude and decides when to stop, the designer succeeds in 9 of 9 trials in a median of two edits given categories of knobs. The model still succeeds with the ungrouped raw knob list and fails when the list is withheld and the change must be described in free prose. The conclusion is a division of labor: a small language model as the namer and the harness as the constructor and stopper.

Methods

The design loop

The system has three parts (Figure 1): a ladder of players that supplies the skill-edge score signal, a simulator that plays them on boards, and the language model that edits the boards. The fixed program that runs the loop, called the harness, simulates, scores, applies edits, and enforces the budget.

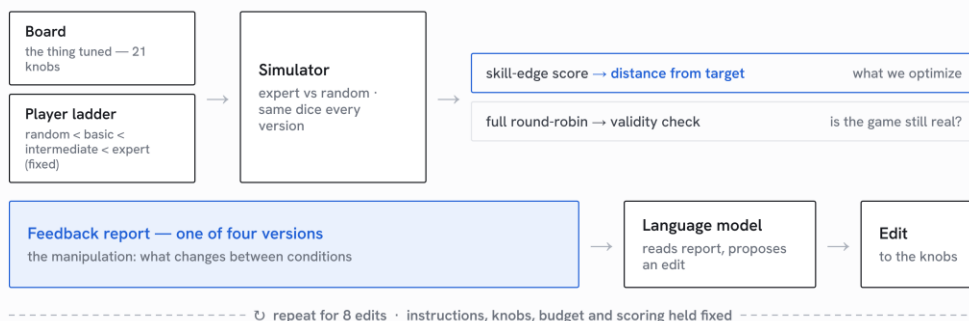


Figure 1. The design loop. Each round, the board and the fixed player ladder are entered into the simulator, which produces the skill-edge score that is optimized and a round-robin validity check. The model reads one feedback report and returns an edit to the knobs, and the cycle repeats for eight edits. The only thing that changes between feedback conditions is the feedback report; while the instructions, knobs, budget, and scoring are held fixed.

The skill-edge objective

The loop optimizes for how much skill matters in game outcomes via the skill-edge objective. It is measured with a single number, the skill-edge score: the share of total wealth that an expert player wins when playing head-to-head against a player that performs random actions (the expert's net worth divided by the two players' combined net worth). The score spans from 0.5 (skill makes no difference; the outcome is pure luck) to 1.0 (the expert always takes everything; the game is a solved puzzle). The designer's job is to steer it to a target of 0.60, with a tolerance zone of 0.57–0.63 so the loop does not optimize against noise. The skill-edge target was determined through three human gameplay rounds on boards tuned to 0.50, 0.60, and 0.70, where 3 of 3 players preferred the 0.60 board. On the standard board the skill-edge score is 0.73 ± 0.04 (120 simulated games; all uncertainties here and below are one standard error). The task is therefore to reduce the score: add luck back until the game is a contestable ~60/40 rather than a near-decided ~73/27.

The score alone is not sufficient because a board can reach 0.60 by destroying the game rather than balancing it. A design is considered degenerate if the skill order breaks (players built to be stronger no longer finish ahead of players built to be weaker), and a degenerate board is rejected.

Verifying the skill ladder

The skill-edge score is only meaningful if the players form a real skill ranking, so the ladder is built, then checked. Each rung is a fixed, hand-written strategy, so playtests are cheap, repeatable, and independent of the model being studied. The ladder is built around growing action spaces. The bottom rung, a random player, chooses blindly among the basic buy actions; a basic player uses the same actions purposefully (greedy buying and building); an intermediate player unlocks trading toward completing color groups, the key Monopoly skill; and an expert player adds an early jail exit plus cash-reserve and building discipline on top of the intermediate player's behavior. Widening the action space this way guarantees the skill ordering by construction: each rung keeps everything the rung below can do and adds options it deploys only where they help, so it can be stronger but not weaker. The players should therefore finish in order: random < basic < intermediate < expert. This is verified with a round-robin competition in which every pair plays. Measured by per-game wealth share, the standings on the standard board rise in order or tie within measurement noise at each rung (random 0.31, basic 0.55, intermediate 0.58, expert 0.56; rank correlation 0.80). The 0.02 gap between the top two has a reason: cash-and-building discipline only pays off against strong opponents, so the intermediate and expert players are close when the opposition is weak.

Ranked by performance against the random player alone, the ladder inverts: trading needs a willing counterparty, and cash discipline is wasted on an opponent who never punishes mistakes. The ranking is therefore validated by the full round-robin, where competent opponents meet, while the skill-edge score keeps the expert-vs-random pairing for what it is meant to measure: the size of the edge skill enables over pure luck.

This validity check is not optimized. It exists to catch a board that reaches the target the wrong way, by collapsing into a game where relative skill does not matter.

The simulator and the editor

Players meet two at a time, with balanced seating and the same dice rolls reused across every board version, so that any difference in outcome comes from the board change, not from luck. This is a variance-reduction technique under which even a few paired games give a usable, though noisy, comparison. The engine itself is a public open-source Monopoly simulator [1], patched to expose the adjustable board; the player ladder, the design loop, and all experiments are original to this project. The language model reads one feedback report and returns its edit as a small structured (JSON) change to the knobs, under fixed instructions and deterministic decoding (no sampling randomness), so a rerun with the same seeds reproduces the trial. The loop runs on two sizes of an open model (Qwen2.5-Instruct, at 1.5 and 7 billion parameters [2]) to test whether any bottleneck is model-size related. The model's exploration is measured behaviorally (which knobs it edits, how large its edits are, how often its edits are rejected as invalid), not from what it claims to be doing.

Matched feedback representation experiment

The central experiment is a set of four feedback reports, designed so that “how much information” and “in what form” vary independently:

1. **No feedback** – the goal only, no report (the ablation).
2. **Numbers table** – the current score with per-round checkpoints, presented as a table.
3. **Progress curve** – a curve of how the wealth share shifts across a single game, showing graphically where the skill gap shifts.
4. **Both** – the table and the curve together.

A random editor (same budget and same knobs, edits chosen blindly) is the negative control. Matching information is not enough; each report must also leave the model an equal ability to act. An early version failed this when only the numbers table named the property groups and edits to the others were silently discarded, so every report here shows the same knob list and logs rejected edits rather than dropping them.

The chain of experiments

The experiments form a chain in which each answer leads to the next question to isolate the bottleneck. The first question is whether more information, or a different format, helps: the four-report comparison on both model sizes to answer the original hypothesis (Findings 1–2). The second is whether the failure is mistaken beliefs, poor judgment, or an inability to act: probes that separate what the model believes from what it can do (Finding 3), and a version of the task with Monopoly removed entirely (a synthetic problem of twelve numbers in which one knob carries the successful outcome) to see whether the model ever tries the right knob (Finding 4). Next is to determine whether the failure is the model’s or the task’s, using reference designers: a scripted rule with no model at all as a positive control (Finding 5), the model’s own sampled candidates scored by the simulator (Findings 6–7), and finally a loop in which the model only names the change while the harness sizes, applies, and stops edits (Finding 8). Each condition is independent across seeds and starting boards, so the trials fan out across cloud GPUs (NVIDIA L4s and A100s); the four-report comparison finished in about 14 minutes, the game-free study (960 trials) in about an hour.

Results

Finding 1 – More information does not help and the curve does not rescue it

What was tested. The four reports × three starting boards × three seeds, on the larger model, evaluated by whether the skill-edge score enters the target zone within the eight edits. The boards stress different traps: the standard board, a double-salary board, and a lopsided board.

What was found (Figure 2). No trial reaches the target zone: across all four reports and all three boards, the average score stays flat near 0.78, about 0.15 above the zone. The no-feedback ablation is as good as or better than the information-rich reports: on the standard board, final error from target is 0.089 for no feedback, versus 0.129 (curve), 0.125 (table), and 0.095 (both). The curve shows no consistent advantage and more feedback narrows the search: the richest report (“both”) explores 3–4 of the 21 knobs, versus 8–12 under the leaner reports – causing fixation, not exploration.

Every report stays near 0.78 skill-edge score — none reach the target zone (larger 7B model; mean $\pm$ SE, 3 seeds)

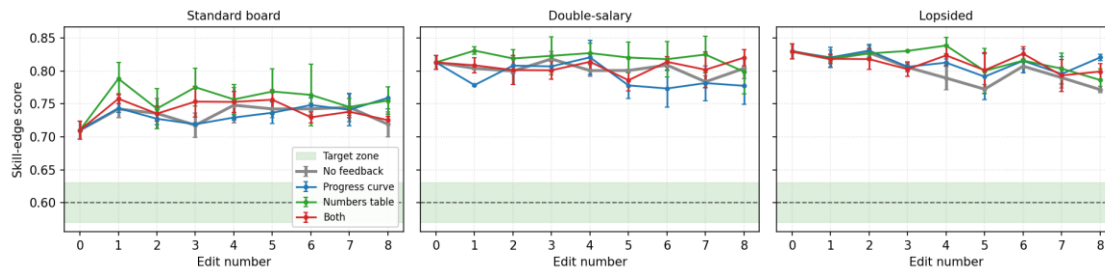


Figure 2. None of the feedback representations lowers the score toward the target: the larger model’s results stay flat near 0.78 on every starting board and never enter the 0.57–0.63 target zone (green band; dashed line: the 0.60 goal; average of three seeds).

What it rules out. That no feedback often does as well as full feedback says information is not the binding variable. The hypothesis is falsified on the larger model: no format can help when the information itself does not.

Finding 2 – The smaller model’s apparent successes are broken games, caught by the validity check

The smaller model returns better error-from-target results, but they are invalid (Figure 3): it reaches them by breaking the game into non-skill-ordered games. In the round-robin, the four players finish nearly tied and the ranking inverts, failing the validity check, so those degenerate trials are excluded. The smaller model also ignores the supplied knob list (30–100% of its edits across trials name knobs that do not exist and are rejected), so the remaining results rest on the larger model.

The smaller model’s improved skill-edge scores are degenerate games (hatched bars failed the validity check)

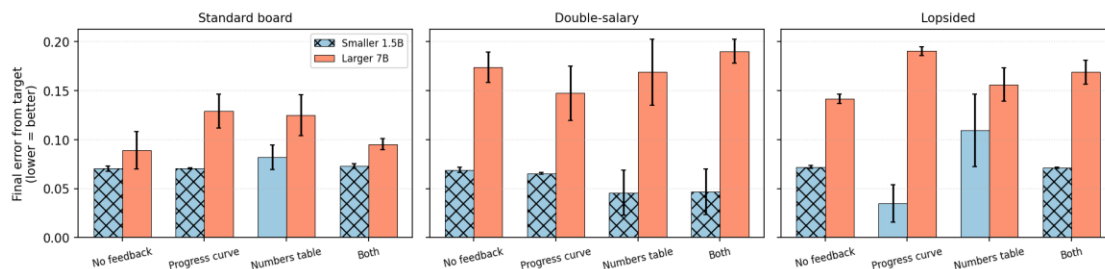


Figure 3. Final error from target by representation and smaller vs larger model (lower error is better). Hatched bars mark the smaller model’s degenerate boards (player ranking inverted) which are flagged and excluded.

Finding 3 – The model knows the right change but cannot execute it

What was tested. With information ruled out, the flat score of Finding 1 could still mean mistaken beliefs about the game, poor judgment of value, or an inability to act on what the model knows. Three probes are used to pull these apart, with the table report used throughout. The first shows the model four finished boards and asks it to pick the best. The second seeds the editing loop from a good board and asks whether the model keeps it. The third seeds the loop from a bad board (the standard board, the same start as the main task) and asks whether the model can build the fix it just recognized. The probes separate recognizing a fix, keeping one, and constructing one, under otherwise matched conditions.

What was found (Figure 4). The model knows the answer and still cannot execute it. Shown four hand-tuned boards – two with reduced rents (the edit that lowers the skill-edge score toward the target) and two distractors – it picks a rent-reduced board in 24 of 24 trials and never a distractor. 23 of the 24 accompanying explanations reason about the goal (“lower rent means a smaller skill gap”) and never about which properties are good in standard Monopoly. Yet seeded from a bad board, it builds the fix in 0 of 5 trials: it edits the near-useless cost knob, adjusts rent in the wrong direction, and oscillates. Seeded from a good board it stays close to the good design’s score in 5 of 5 trials – but keeps editing every round rather than holding the design.

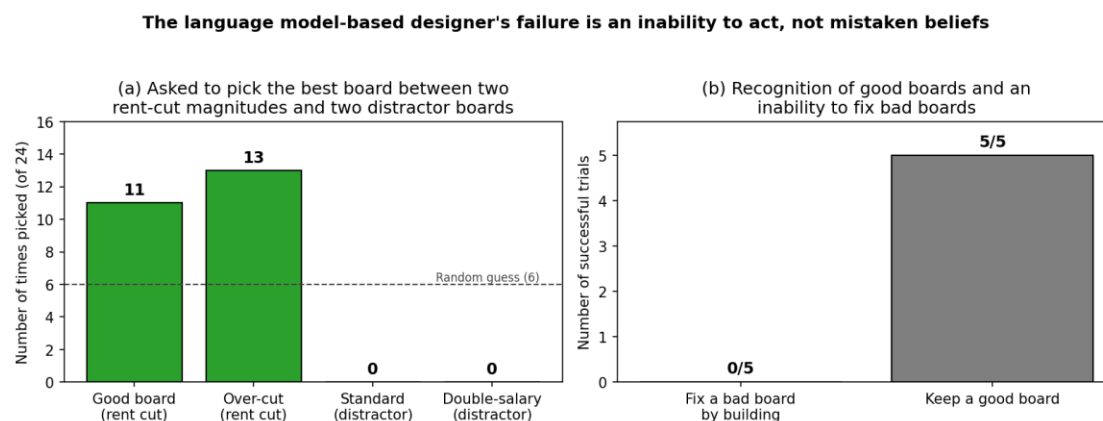


Figure 4. The model knows the right knob but cannot use it. (a) Across 24 picks among four boards, it chooses one of the two rent-cut boards every time (11 picks of the cut sized to reach the target zone, 13 of the larger cut that overshoots past it) and never a distractor. (b) Yet it fixes a bad board in 0 of 5 trials; the 5 of 5 on the right is not recognition of a good board but staying conservatively close to a forgiving starting board.

What it means. Beliefs are ruled out: the model states the correct intent throughout. Its sense of value is miscalibrated: it knows the direction but not the size or the stopping point. The dominant failure mode is the inability to act on its own correct intent.

Finding 4 – The failure mechanism remains after abstracting the game away

What was tested. To check whether any of this is a quirk of the model's Monopoly knowledge, the game was replaced with a synthetic problem of known shape: a function of twelve numbers in which exactly one knob can lower the score to the target, three are dead ends (quick gains that plateau), and eight do nothing. Task structure, the reply format, budget, and scoring are unchanged, but now the right knob is known with certainty, so whether the model ever tried it is directly measurable.

What was found (Figure 5). The failure mode reproduces with no game knowledge in play. The model edits the one knob that matters about as often as a blind editor (8% of its edits, against 9% for random editing, both near 1-in-12 chance rate), and about 90% of its edits go to knobs that provably do nothing. When it does edit the right knob the direction is right (83% of those edits). The score still ends inside the target zone in 24 of 40 trials, but only because the model's habitual cut size happens to match the distance from the start to the wide target basin. The model never says it is done: it declares the design finished in none of those 24 trials, although stopping requires only comparing the displayed score with the target.

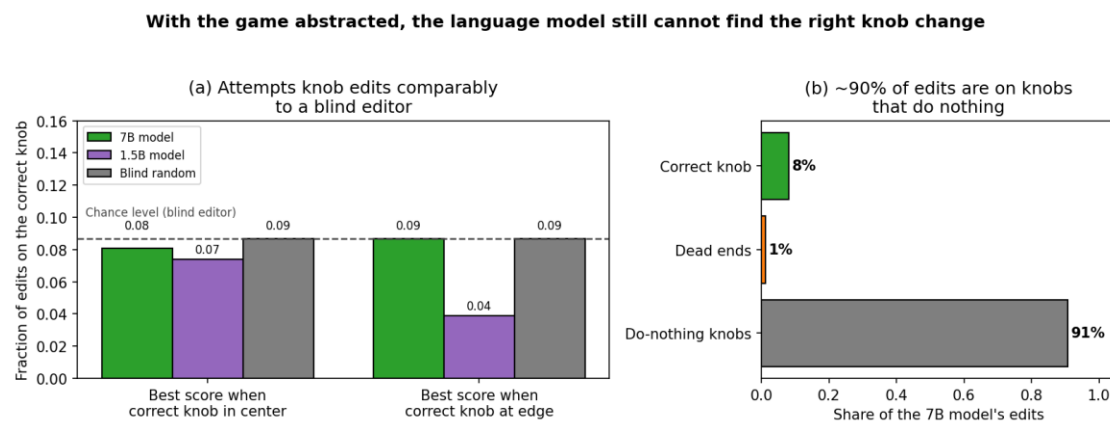


Figure 5. The same failure mode with the game abstracted away. (a) On a synthetic problem where exactly one of twelve knobs can lower the score to the target, both models edit that knob no more often than a blind editor (dashed chance line). (b) About 90% of the larger model's edits go to knobs that provably do nothing.

What it means. The construction failure is not Monopoly-specific; it follows the model onto a problem with no game. The model-reliant loop has no working stopping rule, which is why Finding 8 takes the stopping decision away from the model.

Finding 5 – The task itself is feasible: a scripted rule built from the model's own advice reaches the target

What was tested. The flat lines of Finding 1 can fault the model only if the task is guaranteed solvable under the same noisy measurement and budget. Three scripted designers, none containing a language model, serve as positive controls: one applies the model's own advice from Finding 3 as a fixed rule (cut rents by 20% each round

until the score enters the zone), one binary-searches the size of the rent cut, and one searches blindly, knob by knob.

What was found. The two informed rules solve the task: the fixed-cut rule reaches the zone in 8 of 9 trials (median 6 edits), the binary search in 8 of 9 trials (median 3 edits), and every board they finish on passes the validity check. The blind search succeeds in 1 of 9 trials. The model's own one-sentence advice plus a fixed step size is sufficient under this budget and noise, and mechanical search without it is not: the model fails to execute its own stated advice in a feasible task.

Finding 6 – The right edit exists in the model's own sampled candidates and is lost at the moment of commitment

What was tested. Inability to construct the edit could mean the productive edit never appears among the model's outputs or that it appears and is discarded at commitment. From a fixed mid-loop state, the model was asked for 50 candidate edits at each of two sampling temperatures; every candidate was scored in the simulator under the same dice sequence and compared with the single edit committed under deterministic decoding.

What was found. Improving edits are present but the committed answer discards them: 26–40% of the sampled edits improve the board. The most common proposal is the same rent cut as the scripted rule of Finding 5. The committed answer is worse than 92% of its own samples at the higher temperature and 82% at the lower. The written rationale says to “reduce the cost and rent [...] to increase randomness,” and the edit it emits raises rent to 1.2x. No single sampled edit reaches the zone in one step, implying success requires compounding the cut across rounds, which the scripted rule does and the model does not.

Finding 7 – Better selection and more edit attempts do not repair construction

What was tested. Finding 6 suggests a repair: each round, sample many edits from the model, score each in the simulator, and keep the best, applied only if it beats the current board under the same dice sequence. The sample size, 22 per round, is read off Finding 6's data: beyond 22 candidates, extra samples negligibly improve the best edit found. A second experiment doubles the edit budget from 8 to 16.

What was found. Picking the best of the 22 reaches the zone in 1 of 9 trials. The score does decrease in 8 of 9 trials, so improving candidates keep being found, but it descends at approximately half the scripted rule's pace, because the picked edits are small or often go to knobs that barely affect the score. Doubling the budget does not help: 1 of 9 trials succeeds again, and the added rounds actively hurt. The eight trials that never reach the zone improve the score by 0.041 on average over the first eight edits, then give back 0.021 over the next eight. Six of those eight regress: a quarter to a third of proposals raise rent back up, and some slip past the noisy acceptance test. The deficit is not the number of attempts or the absence of good candidates but a poor sense of magnitude: the model's proposals do not adapt in size as the design improves.

Finding 8 – The designer succeeds once the model splits responsibility with the harness

What was tested and found (Figure 6). The final experiments transfer responsibility for the step the model fails at – constructing and sizing the edit – to the harness, in three tested designs. In name-and-measure, the model names only the knob family and the direction (“rent, down”); the harness tries three preset magnitudes of that change plus the current board under the same dice sequence, keeps the best, and stops when the score enters the zone. It places 9 of 9 trials in the target zone in a median of two edits – faster than the scripted rule’s median of six – and every final board passes the validity check. The model names “rent, down” in all 25 rounds those nine trials used. The preset magnitudes include the large cuts that free-form generation never produced.

The uncategorized design reruns name-and-measure with the full 21-knob list exactly as the simulator exposes it (no hand grouping). The final test runs with no knob list at all, the model describing the change in free prose that the harness maps to knobs by string matching. With the raw list, the model names the rent family in all 40 rounds, groups the knobs into sets unprompted, rebuilding the curated families on its own (median size six knobs), and succeeds in 7 of 9 trials (median 2 edits). With no list, it succeeds in 0 of 9 trials: every prose description maps to a valid knob, but the descriptions collapse to one knob at a time, creating too weak a cut to compound in eight edits, and the direction is inconsistent (17% of phrases raise rent).

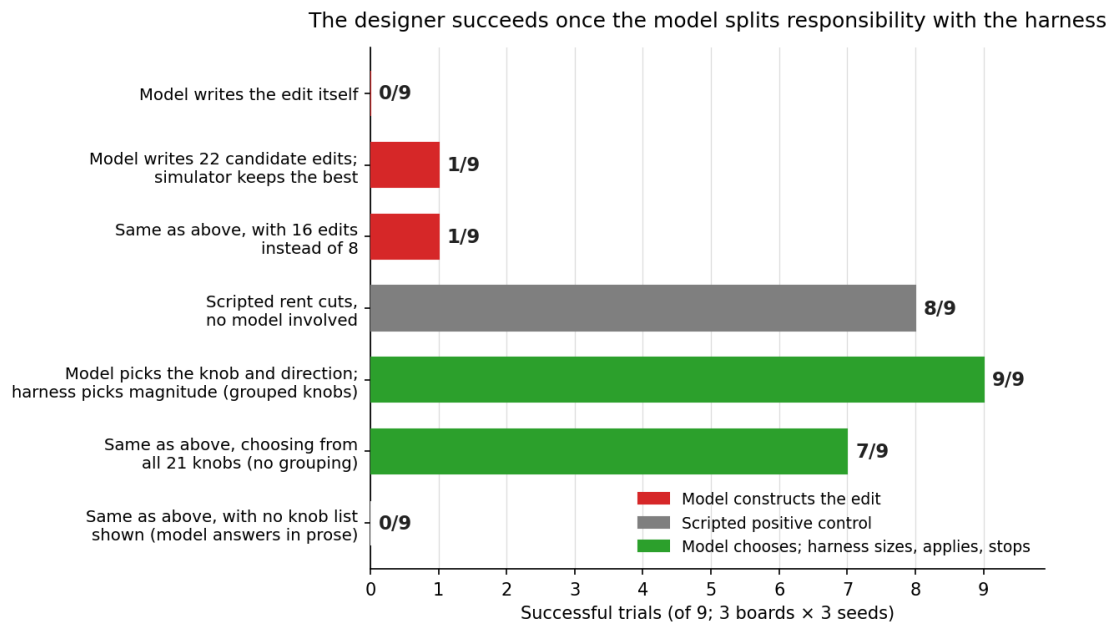


Figure 6. Trials reaching the target zone for each division of labor design (of 9; three boards × three seeds, eight-edit budget, validity-checked). Every design in which the model writes the edit fails (top, red); every design in which it only names while the harness sizes, applies, and stops succeeds (green), except the free-prose variant with the knob list withheld. The scripted rule (grey) is the no-model positive control.

What the model's knowledge is worth in the working design. The working design still leans on the model for deciding which knob family the harness should measure first. A final experiment replaced that family ranking with random orderings and compared with an oracle that told the right family in advance. The model matches the oracle on Monopoly, succeeding in 9 of 9 trials and testing rent first, while random orderings reach the zone in 20 of 27 trials but act on the rent family in only 11 of 27. The model's knowledge enables reliability on the right knob, but the loop survives without it, at least on a small board game. No ordering produces any invalid boards.

Discussion

The bottleneck for a small language-model game designer is not information or its format, a result that held across two model sizes, a real game, and a game-free ablation. The bottleneck is constructing the change and committing to it, especially at appropriate magnitudes. The original hypothesis fared accordingly: its negative half ("more information will not help") was confirmed, and its constructive half ("the curve will beat the table") was falsified, because no representation can fix a failure that is not informational.

The study found the repair is to split responsibility by capability using a harness. The model's capabilities (naming the right knob, recognizing the boards that embody the fix) are the inputs a classical optimization loop needs; the capability it lacks (sizing, applying, and stopping an edit) is what such a loop does well. The repair does not remove the failing step; instead it transfers responsibility for it: the language model names and the harness constructs, measures, and stops. Using the harness, the same model that succeeded in 0 of 9 trials under every format succeeds in 9 of 9 trials in a median of two edits. It does so while simulating only four candidate boards per round, against the 22 sampled edits per round that screening the model's own free-form proposals required (Finding 7).

Two behavioral facts shape the working design. First, the harness must own the stopping decision: across the entire project the model declared the design finished zero times, even in trials that ended inside the target zone (Finding 4). Second, the model must see the knobs, because its competence is reading and ranking what is displayed, not imagining structure. The same chooser that is flawless over a visible 21-knob list produces, with no list shown, prose with inconsistent direction. Show the model the knobs; do not make it imagine them. Whether these rules are properties of small-model optimization loops in general, rather than of this game, can only be extrapolated from this study.

Limitations. Control design: Finding 4's synthetic problem to isolate whether the model has any sense of an optimum fell to geometry confounds. **Model size:** the load-bearing conclusions are on the 7-billion-parameter model; the smaller model is generally excluded because it both ignores the supplied knob list and reaches its numbers by breaking the game. Whether the construction bottleneck eases at much larger sizes or with LLMs with advanced reasoning is untested. **Breadth:** the real-game

evidence leans on Monopoly and cannot be readily generalized to different styles of multiplayer games.

Conclusion

A small language model placed in a game designer's role fails in a specific way: it knows the right change, states it, recognizes boards that embody it, but cannot construct, size, or commit the edit, no matter what feedback it receives, in what format, or how many tries it gets. The failure mode remains after the game itself is removed. The repair is architectural, not informational: transfer responsibility for construction to the harness. A repaired designer is developed in which the model only names the knob and the edit direction, while the harness determines the edit magnitude, applies it, and decides when to stop. This architecture reliably and quickly solves a task that the same model alone cannot. For automated playtesting loops built on small models, this study suggests three operating rules: display the knob list to the model, let it name the change, and leave construction, magnitude, and stopping to the harness.

References

1. gamescomputersplay. monopoly.
<https://github.com/gamescomputersplay/monopoly>
2. Qwen Team (2024). Qwen2.5 Technical Report. arXiv:2412.15115